Counting Sort

Distribution-Based Sorting & Frequency Analysis

Module 14: Non-Comparative Algorithms

Series: CodeHive Advanced DSA

01. Beyond Comparison

Most sorting algorithms (Bubble, Quick, Merge) are **Comparison Sorts**; they decide order by comparing two elements. **Counting Sort** is fundamentally different. It uses the frequency of values to determine their final positions.

The Core Idea

Instead of asking "Is A greater than B?", Counting Sort asks "How many times does the value 5 appear?". By talling every occurrence in a special 'counting array', the algorithm can reconstruct the list in perfect order without a single relative comparison.

Efficiency Trick: Because it doesn't compare, it can theoretically break the lower bound of $O(n \log n)$ that limits comparison-based sorts.

02. Critical Prerequisites

Counting Sort is highly specialized. It is incredibly fast, but only under specific architectural conditions:

1. Non-Negative Integers: Values are used as indices in the counting array. Negative numbers would result in "Index of Out Range" errors.

2. Limited Range (k): If you have 10 numbers to sort (n=10) but one of them is 1,000,000,000 (k=1B), you must create a counting array with 1 billion slots. This is a massive waste of memory.

3. Integer Only: It cannot naturally sort strings, floats, or objects without an additional mapping layer, because floating point values cannot be array indices.

03. Execution Trace

Target Array: `[2, 3, 0, 2, 3, 2]`

Step 1: Initialization

Find the max value (3). Create a Counting Array of size 4: `[0, 0, 0, 0]`

Step 2: Tally Frequency

- Value 2 appears 3 times.
- Value 3 appears 2 times.
- Value 0 appears 1 time.
- Value 1 appears 0 times.

Counting Array: `[1, 0, 3, 2]`

Step 3: Reconstruction

Output 0 (one time). Output 2 (three times). Output 3 (two times).
Result: `[0, 2, 2, 2, 3, 3]`

04. Python Implementation

The Python implementation requires calculating the range dynamically to ensure the counting array is the correct size.

```
def countingSort(arr):
    if not arr: return arr

    max_val = max(arr)
    # Create counting array with zeros
    count = [0] * (max_val + 1)

    # Phase 1: Count frequencies
    while len(arr) > 0:
        num = arr.pop(0)
        count[num] += 1

    # Phase 2: Reconstruct sorted array
    for value, freq in enumerate(count):
        while freq > 0:
            arr.append(value)
            freq -= 1

    return arr
```

05. Big O Analysis

The time complexity is expressed as a sum of the input size (n) and the range of values (k).

$$O(n + k)$$

Performance Scenarios:

- **Best Case:** When $k pprox n$ or $k < n$, the algorithm is effectively **Linear $O(n)$**. This is faster than Quicksort.
- **Worst Case:** When k is much larger than n (e.g., $k = n^2$), the complexity becomes **$O(n^2)$** and uses excessive memory.
- **Space Complexity:** $O(k)$ - The memory required depends entirely on the maximum value in the input.

06. Practical Summary

Counting Sort is an "exotic" algorithm. It isn't a general-purpose tool, but in the right context, it is unbeatable.

When to use Counting Sort:

- When sorting **phone numbers** or **zip codes** (fixed range).
- When sorting **exam scores** (range 0-100) for a million students.
- As a subroutine for **Radix Sort**.

CODEHIVE ACADEMY